

이번 주차에는 MobileNetV2 Layer 8의 **depthwise 3×3 convolution**을 RTL로 구현했다. 지난주에 만든 **일반 MAC 곱셈기(pointwise, 채널을 섞어 곱-누적하는 구조)**와 근본적으로 무엇이 다른지, 그리고 depthwise에서만 등장하는 **zero padding**을 왜, 어떻게 처리했는지를 핵심만 추려 정리한다.

i 이 글에서 "일반 MAC(일반 mul)"이란

지난주 pointwise처럼 **입력을 그대로 weight와 곱해 누적**하는 표준 곱셈-누적 구조를 말한다. 공간(이웃 픽셀) 개념이 없고 채널 방향으로만 더한다. 이걸 기준으로 두고 depthwise가 무엇을 더 해야 하는지를 본다.

두 연산을 한눈에

항목	일반 MAC (pointwise 1×1)	Depthwise 3×3
곱하는 대상	같은 위치의 여러 채널	같은 채널의 3×3 이웃 픽셀
채널 방향 합산	있음 (채널을 섞음)	없음 (채널마다 독립)
공간 이웃	없음 (자기 자리 하나)	3×3 window 필요
Zero padding	불필요	필수 (가장자리 처리)
입력 공급	넓은 word 한 번에 read	라인버퍼 2개로 스트리밍
병렬 축	채널 (한 word에 여러 채널)	채널 (채널이 독립이라)

→ 한 줄로 요약하면, **일반 MAC은 "채널을 섞는" 연산**이고 **depthwise는 "공간을 보는" 연산**이다. 이 차이 하나에서 padding, 라인버퍼, window 같은 depthwise의 모든 추가 장치가 따라 나온다.

차이 1 — 채널을 섞느냐, 채널이 독립이나

일반 convolution(그리고 pointwise)은 한 출력값을 만들 때 **모든 입력 채널을 더한다**.

출력[oc] = $\sum(ic)$ 입력[ic] × weight[oc][ic] ← 채널 방향 합산

반면 depthwise는 채널을 **전혀 섞지 않는다**. 채널 0은 채널 0끼리, 채널 383은 채널 383끼리만 계산한다.

입력 ch0 — kernel0 — 출력 ch0
 입력 ch1 — kernel1 — 출력 ch1
 :
 입력 ch383 — kernel383 — 출력 ch383

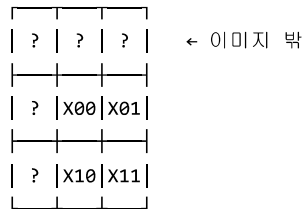
→ 채널이 완전히 독립이라는 점이 하드웨어에서 큰 이점이 된다. **채널 방향을 그대로 병렬 축으로 쓸 수 있어서**(PAR_CH), 카운터·마스크 같은 제어는 한 벌만 두고 연산기(라인버퍼+DSP)만 채널 수만큼 복제하면 된다.

차이 2 — Zero Padding은 왜 depthwise에서만 필요한가 (핵심)

일반 MAC(1×1)은 출력 한 픽셀이 **자기 위치 하나만** 본다. 이웃을 안 보니 이미지 밖을 참조할 일이 없고, 그래서 padding이 필요 없다.

depthwise는 3×3이라 출력 한 픽셀이 **자기 주변 8칸까지** 본다. 그런데 가장자리 픽셀은 이웃 일부가 이미지 밖에 있다.

바깥(존재 안 함)



출력 (0,0) 을 만들 때 window

이 "바깥값"을 **0으로 채워 넣는 것이 zero padding**이다. 그래야 3×3을 적용해도 출력 크기가 입력과 같은 14×14로 유지된다. (padding이 없으면 출력이 12×12로 줄어든다.)

검증 기준으로 삼은 유효 tap 개수는 위치마다 다음과 같다.

corner(모서리) : 유효 4칸 edge(가장자리) : 유효 6칸 interior(내부) : 유효 9칸

→ 일반 MAC엔 없던 "가장자리 문제"가 3×3 window 때문에 생기고, 그 해법이 zero padding이다. 입력·weight를 전부 1로 넣으면 출력이 corner 4 / edge 6 / interior 9로 나와야 padding이 맞은 것이다.

Zero Padding을 어떻게 구현했나 — 두 가지 방식

같은 결과를 내는 두 방식을 비교 대상으로 잡았다. 결과는 완전히 동일해야 하고, 차이는 연산량·전력뿐이다.

※ 방식 1 — window에 0을 넣고 9탭 항상 계산 (채택·구현)

padding을 데이터 문제로 본다. window에 0을 채워 넣고, 곱셈기 9개는 가장자리인지 아닌지 신경 쓰지 않고 늘 9탭을 곱한다.

핵심은 스트리밍 구조다. 입력을 1픽셀/클럭으로 흘리면 현재 행은 입력 그 자체이고, 위 두 행만 라인버퍼 2개로 지연시키면 3×3의 세 행이 동시에 모인다.

```
input -> [ line_buf_1 ] -> [ line_buf_2 ]
      |           |           |
      bot         mid         top
(현재행)   (1행 전)   (2행 전)
```

여기서 0을 넣는 위치는 라인버퍼 출력 3곳에 둔 mux 3개뿐이다.

```
wire bot_in = (col_zero | bot_zero) ? '0 : bot;
wire mid_in = (col_zero | mid_zero) ? '0 : mid;
wire top_in = (col_zero | top_zero) ? '0 : top;
```

mux 3개로 상·하·좌·우 4방향 padding이 전부 처리된다. 좌우가 하나로 되는 이유가 재미있는데, 더미 열에서 넣은 0이 그 순간엔 window 오른 쪽(우측 padding)에 있다가 2클럭 뒤 왼쪽으로 밀려와(좌측 padding) 양쪽을 검하기 때문이다.

이걸 위해 실제 14×14 이미지를 **15×15 스트림 프레임**으로 돌린다. 더미 행·열이 1개씩 필요한 이유는, window가 지금 들어온 픽셀을 우하단으로 잡아 스트림보다 2칸 뒤에서 완성되기 때문이다. 그래서 채널당 15×15 = 225 clk (유효 출력 196개)이 든다.

✍ 왜 16×16이 아니라 15×15인가

왼쪽 padding 열을 더미 열 하나가 검하기 때문에 padding 칸을 따로 다 두지 않아도 된다. 덕분에 채널당 31 clk을 아꼈다.

※ 방식 2 — 0인 게 확정된 행은 연산 자체를 생략 (미착수, 비교용)

top_zero / bot_zero 는 **행 단위 조건**이라 한 번 켜지면 그 행 15클럭 내내 유지된다. 즉 출력 행 0~13을 계산하는 구간에선 곱셈기 3개가 무조건 0을 곱한다 → 아예 눌러도 된다.

다만 냉정하게 볼 점이 있다.

- 곱셈 9개가 **병렬**이라 3개를 빼도 **사이클(throughput)**은 **안 줄어든다**. 이득은 오직 전력.
- 방식 1의 mux도 이미 해당 tap을 0으로 만들어 곱셈 결과가 0이 되므로 레지스터 토글이 줄어든다. **방식 1이 이미 이득의 상당 부분을 가져간다**.

→ 그래서 "방식 2의 **순수 추가** 전력 이득이 측정 가능한 수준인가"가 이 비교의 진짜 질문이다. 이걸 게이트 수로 추정하지 않고 **SAIF/VCD로 실측**해서 판단한다.

데이터패스 비교

※ 일반 MAC (pointwise) — 넓은 word 병렬 곱셈

한 주소에 여러 채널을 붙여 넓은 word로 read하고, 곱셈기를 병렬로 깔아 곱한 뒤 누적한다. 입력을 **그대로** 곱하면 되고 공간 개념이 없어 구조가 단순하다.

BRAM(넓은 word) → 채널 레인 슬라이스 → 곱셈기 병렬 → 덧셈 트리 → 누적

※ Depthwise — 라인버퍼 스트리밍 + DSP 캐스케이드

3×3 이웃이 필요하므로 라인버퍼로 세 행을 모으고, 곱셈-누적은 **DSP 3체인 × 3탭 systolic**으로 구성했다. 부분합이 $PCOUT \rightarrow PCIN$ 으로 옆 DSP에 1클럭씩 내려가며 한 행(3탭)을 누적하고, 세 행의 결과를 마지막에 더한다.

```

top_in → DSP → ACOUT → DSP → ACOUT → DSP → top_out 1
                |           |
                | PCOUT →  | PCOUT →
mid_in → (동일 3탭 체인)   → mid_out —————→ [+reg] → data_out
bot_in → (동일 3탭 체인)   → bot_out —————→
  
```

- 캐스케이드가 **열 방향 shift**를 **공짜로** 해줘서 별도 window 레지스터·skew 지연선이 사라졌다.
- IFM read는 **픽셀당 정확히 1회**(이론적 최소). 라인버퍼가 재사용을 담당한다.
- OOC 합성 기준 자원: **DSP 9 / LUT 41 / FF 89 / LUTRAM 32**. LUT 41이 작다는 건 fabric에 큰 가산 트리 없이 DSP 내부 가산기를 썼다는 뜻이고, LUTRAM 32는 라인버퍼가 SRL16E로 추론됐다는 증거다.

→ 일반 MAC은 "**넓게 읽어 병렬로 곱하기**", depthwise는 "**좁게 흘리며 이웃을 모으기**" 다. 같은 곱셈-누적이라도 공간을 봐야 하는 순간 데이터 공급 구조가 통째로 달라진다.

연산량 비교

	일반 MAC (pointwise 1×1)	Depthwise 3×3
채널당 곱셈	$64 \times 196 = 12,544$	$9 \times 196 = 1,764$
전체 곱셈	4,816,896	677,376

→ depthwise는 채널을 안 섞기 때문에 전체 곱셈이 일반 MAC의 **약 1/7**로 줄어든다. 연산을 **pointwise(채널 방향)**와 **depthwise(공간 방향)**로 **쪼개 연산량을 줄이는 것**이 MobileNet separable convolution의 핵심이고, 이번 구현이 그 depthwise 절반을 담당한다.

검증

입력을 프레임 위치 $n = \text{row} \times 15 + \text{col}$ 로 넣으면(weight는 전부 1) window 안의 값이 곧 주소가 되어, 출력은 "범위 안 3×3 이웃의 n 값을 더한 것"과 같아야 한다. testbench가 이 golden과 LAT=7 지연을 맞춰 자동 비교한다.

```
LAT      : 7
checked  : 392 / 392
errors   : 0
```

→ 유효 출력 392개(2프레임 × 196) 전부 **errors 0**. corner 4 / edge 6 / interior 9의 padding 동작과 3×3 누산이 모두 맞음을 확인했다.

나의 질문

Q. 일반 MAC엔 없던 라인버퍼·window·padding이 depthwise에선 왜 한꺼번에 등장하는가?

A. 전부 "3×3 이웃을 본다"는 한 가지 요구에서 파생된다. 이웃을 보려면 서로 다른 행을 동시에 가지고 있어야 하고(→ 라인버퍼), 그 3×3을 담을 곳이 필요하며(→ window/DSP 체인), 이웃이 이미지 밖으로 나가는 가장자리를 메워야 한다(→ zero padding). 일반 MAC(1×1)은 자기 위치 하나만 보므로 이 셋이 전부 필요 없다. **연산이 "공간"을 보는 순간 데이터를 시간축으로 정렬하는 장치들이 따라온다.**

결론 : 일반 MAC은 채널을 섞어 자기 위치 하나만 곱하므로 padding이 없지만, depthwise 3×3은 채널 독립으로 공간 이웃을 보기 때문에 라인버퍼 스트리밍과 zero padding이 필수다. padding은 window에 0을 넣어 9탭을 항상 계산하는 방식 1(mux 3개로 4방향 처리, 15×15 프레임)로 구현해 검증 392/392 errors 0을 통과했고, 채널을 안 섞는 덕에 전체 곱셈은 일반 MAC의 약 1/7이다.